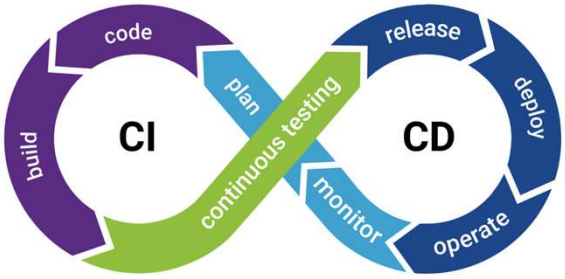
Terraform Fundamentals

Instructor: Dyego Souza
Findmore Academy
https://www.linkedin.com/in/dyegodesouza/
ramehx@yahoo.com.br

SESSION 5: IAC INTEGRATION WITH GITHUB ACTIONS AND CI/CD PIPELINES

Introduction to CI / CD

- **Continuous Integration (CI)**
- Practice of frequently integrating code changes into a shared repository
- Each change is automatically:
  - validated
  - tested
  - checked for errors
- **Continuous Deployment / Delivery (CD)**
- Practice of automating the release process
- Ensures changes can be:
  - safely deployed (Continuous Delivery)
  - automatically deployed to production (Continuous Deployment)
- **Core Goal**
- Deliver changes **faster, safer, and more consistently**



Fonte: https://financesonline.com/devops-best-practices-everyone-should-follow/

FINDMORE ACADEMY

Interno

Why Use CI/CD with Terraform?

Consistency
- Infrastructure changes follow the same process every time

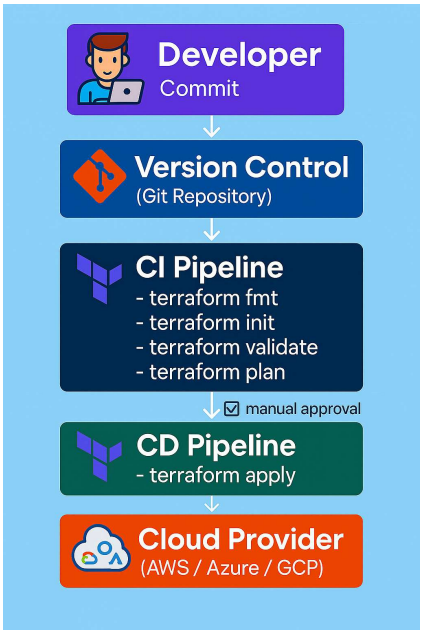
Early Validation
- Catch errors using terraform fmt, validate, and plan

Auditability
- Every change is tracked through version control

Reduced Risk
- Plans reviewed before apply (pipeline approval mechanisms)
- Fewer manual changes

Team Collaboration
- Enables safe, parallel work on infrastructure

Infrastructure changes become **reviewable, testable, and repeatable**



FINDMORE ACADEMY

Interno

GitHub Actions

- **Workflows in GitHub Actions**

Workflows define automated processes triggered by specific events in a repository for continuous integration and deployment.

- **Jobs Execution**

Jobs are individual tasks within workflows, running sequentially or in parallel on specified runners.

- **Runners**

Runners are servers that execute jobs; they can be GitHub-hosted or self-hosted machines.

- **Triggers**

Triggers initiate workflows based on repository events like push, pull request, or schedule.

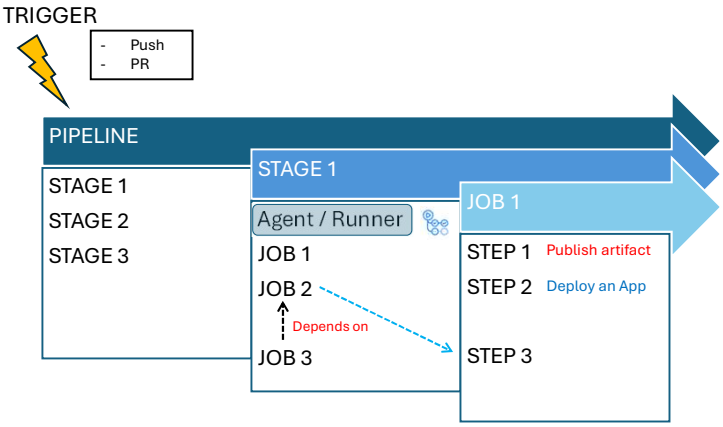
GitHub Actions

FINDMORE ACADEMY

Interno

70

Introduction to “Pipeline as a code”



https://www.jetbrains.com/teamcity/pipelines/

Main Characteristics:

- Defined in source code (YAML, etc.), stored in version control
- Declarative pipeline stages and jobs
- Version controlled and auditable
- Integrated with application code
- Automated execution in CI/CD environments
- Extensible and reusable templates

Advantages:

- Consistent and repeatable processes
- Enables collaboration and code reviews
- Full audit trail for compliance
- Rapid recovery and migration
- Reduced manual errors
- Accelerates delivery cycles
- Scalable across teams and projects

FINDMORE ACADEMY

Interno

71

GITHUB ACTIONS CI PIPELINE

- **on:** defines the triggers (in this case, pull request and push on main).
- **jobs:** defines one or more jobs.
- **runs-on:** specifies the runner and environment for the job.
- **steps:** lists each action (checkout, setup, install, test, build, etc.).
- **uses:** calls reusable GitHub Actions (official or custom).
- **run:** executes shell commands.

```yaml
name: Terraform CI/CD

on:
  pull_request:   # Run CI on pull requests (plan only)

  # Run CD on pushes to main (apply)
  push:
    branches:
      - main

jobs:
  # Terraform PLAN (CI Phase)
  terraform-plan:
    name: Terraform Plan
    runs-on: ubuntu-latest

    steps:
      # Checkout repository code
      - name: Checkout code
        uses: actions/checkout@v4

      # Install the specified Terraform version
      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: ${{ env.TF_VERSION }}

      # Initialize Terraform (downloads providers, configures backend)
      - name: Terraform Init
        run: terraform init

      # Enforce Terraform formatting standards
      - name: Terraform Format Check
        run: terraform fmt -check

      # Validate Terraform configuration files
      - name: Terraform Validate
        run: terraform validate

      # Generate an execution plan (no infrastructure changes)
      - name: Terraform Plan
        run: terraform plan
```


GITHUB ACTIONS DEPLOYMENT PHASE (CD)

- Runs on **pull_request** to main when files in src/ or workflow configs change
- **Needs:** creates a sequential and dependency logic so the next stage only runs if the previous stage succeed
- **Environment:** defines the environment and creates approval gates

https://github.com/actions

```yaml
# Terraform APPLY (CD Phase)
terraform-apply:
  name: Terraform Apply
  runs-on: ubuntu-latest

  # Apply only runs after a successful plan
  needs: terraform-plan

  # Extra safety: only apply on main branch
  if: github.ref == 'refs/heads/main'

  environment:
    name: production
    # This enables manual approval if environment protection
    # rules are set in the GitHub repository settings

  steps:
    # Checkout repository code
    - name: Checkout code
      uses: actions/checkout@v4

    # Install the same Terraform version used in the plan
    - name: Setup Terraform
      uses: hashicorp/setup-terraform@v3
      with:
        terraform_version: ${{ env.TF_VERSION }}

    # Re-initialize Terraform to ensure backend consistency
    - name: Terraform Init
      run: terraform init

    # Apply the previously reviewed changes
    - name: Terraform Apply
      run: terraform apply -auto-approve
```

Service Principal Identity in CI/CD

What Is a Service Principal?

- A non-human identity in Azure Active Directory
- Used by applications, automation, and CI/CD pipelines
- Authenticates and authorizes access to Azure resources

Role in Terraform CI/CD Pipelines

- CI/CD pipeline uses a Service Principal to authenticate to Azure
- Terraform uses this identity to:
  - Create, update, and delete Azure resources
  - Access remote state (e.g., Azure Storage backend)
- Permissions are controlled via Azure RBAC roles (Ex: Contributor, Reader, or custom roles)

Authentication Flow

1. CI/CD pipeline runs Terraform
2. Terraform authenticates using the Service Principal
3. Azure validates identity and permissions
4. Infrastructure changes are applied securely

Azure Service Principal

Fonte: https://argv.cloud/blog/2024/azure-service-principal-automation/

FINDMORE ACADEMY

Interno

Secret Management & GitHub Environment Variables

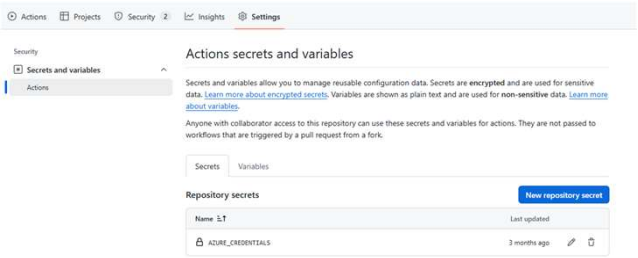
When you create a Service Principal in Azure, it outputs credentials that must be securely stored and used by the CI/CD pipeline to authenticate and deploy Terraform infrastructure.

Service Principal Credentials

- Client ID
- Client Secret
- Tenant ID
- Subscription ID

Best Practices

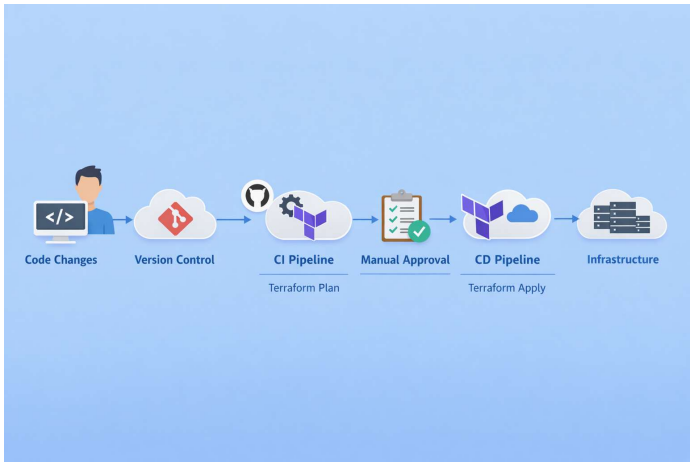
- Store credentials as **GitHub Secrets**
- Never commit secrets to the repository
- Inject secrets into the pipeline as **environment variables**



FINDMORE ACADEMY

Interno

Lab 5: CI/CD Pipeline with Terraform and Github Actions



Interno

Conclusion

Terraform Core Concepts

Learned how Terraform uses declarative configuration, providers, and state to manage infrastructure safely.

State, Dependencies, and Locking

Explored state management, dependency graphs, locking, and provider version control.

Modules, Inputs, and Outputs

Understood how to structure reusable modules and pass data between parent and child modules.

Expressions and Resource Iteration

Applied count, for_each, for, and conditionals to build flexible and maintainable configurations.

CI/CD Integration for Infrastructure as Code

Implemented Terraform in CI/CD pipelines using plans, approvals, secure identities, and secret management.

Interno

Next steps

Practice with Real Environments
Apply Terraform to real cloud resources using dev, staging, and production environments.

Deepen Module Design Skills
Build reusable, versioned modules and publish them internally or to a private registry.

Enhance CI/CD Pipelines
Add approvals, environment promotions, and security checks to Terraform pipelines.

Explore Advanced Terraform Topics
Learn about workspaces, policy as code (Sentinel / OPA), testing, and cost controls.

Interno

QUESTIONS ?

Interno

THANK YOU!

CONTENT:
FM ACADEMY
INSTRUCTOR: DYEGO SOUZA
RAMEHX@YAHOO.COM.BR
HTTPS://WWW.LINKEDIN.COM/IN/DYEGODESOUZA/

Interno